OCR Word Search Solver

Last Report – 2nd Defense

Date: December 14th, 2025

Team Members:

Eliott	– Image preprocessing & manual rotation
Dawid	– Manual localization & letter splitting
Thomas	– Solver algorithm (C)
Tiago	– Neural network (Proof of Concept)

Contents

1 Introduction **3**

2 Objectives of the First Defense **5**

3 Team Organization and Planning **7**

4 Image Preprocessing (Eliott) **9**
4.1 Objective . . . 9
4.2 Personal Experience and Steps Implemented . . . 9
4.3 Complexity Analysis . . . 10
4.4 Example . . . 11
4.5 Integration with the OCR Pipeline . . . 11
4.6 Personal Reflection . . . 12

5 Manual Localization & Letter Splitting (Dawid) **13**
5.1 Purpose of the Module . . . 13
5.2 Implementation Process . . . 13
5.2.1 Grid Detection with `find_grid()` . . . 13
5.2.2 Grid Line Removal with `remove_grid_lines()` . . . 15
5.2.3 Letter Extraction with `find_all_letters()` . . . 16
5.2.4 Word List Extraction . . . 17
5.3 Multithreaded Architecture Overview . . . 17
5.3.1 Responsibilities of the Main Thread . . . 17
5.3.2 Task Delegation to Worker Threads . . . 18
5.3.3 Worker Thread Processing . . . 18
5.3.4 Synchronization and Thread Safety . . . 19
5.3.5 Performance Impact and Hardware Considerations . . . 19
5.4 Comparison Method . . . 19
5.4.1 Letter Detection and Recognition Process . . . 19
5.4.2 Choice of Comparison Method . . . 20
5.4.3 Classic Comparison Mode . . . 20
5.4.4 Neural Comparison Mode . . . 21
5.4.5 Design Trade-offs . . . 21
5.5 Challenges and Solutions . . . 22
5.6 Various Optimizations . . . 22
5.6.1 Resource Loading and Initialization Strategy . . . 22
5.6.2 Bitmap Dataset Loading . . . 22
5.6.3 Neural Network Loading and Cloning . . . 23
5.6.4 Resource Lifetime Management . . . 23

5.6.5 Design Benefits . . . 23
5.6.6 Impact of Optimizations on Performance . . . 23
5.7 Comparison with the Program Presented in the First Defense . . . 24
5.7.1 Performance Evaluation and Comparison . . . 24
5.7.2 Notes on the Results . . . 24
5.7.3 Observations . . . 24
5.8 Demonstration Example . . . 25
5.9 Integration Within the OCR Pipeline . . . 25
5.10 Next Development Steps . . . 26
5.11 Reflection . . . 26

6 Word Search Solver (Thomas) 27
6.1 Objective . . . 27
6.2 Personal Work and Experience . . . 28
6.3 Implementation and Technical Additions . . . 28
6.3.1 Resource Loading: Textures for Letters and UI Elements . . . 29
6.3.2 Grid Rendering: Scaling from Grid Coordinates to Pixel Coordinates 29
6.3.3 Word List Rendering using `SDL_ttf` . . . 30
6.3.4 Highlighting Found Words on the Grid . . . 30
6.3.5 Graphical Style and Visualization Choices . . . 30
6.4 Challenges and Solutions . . . 31
6.5 Integration with the OCR Pipeline . . . 32

7 Neural Network Proof of Concept (Tiago) 33
7.1 Introduction . . . 33
7.2 System Architecture . . . 33
7.3 Algorithms Used . . . 34
7.3.1 Neural Network Forward and Backpropagation . . . 34
7.3.2 Logical Operation Learning . . . 34
7.3.3 Learned Similarity Network . . . 34
7.4 Final System Behavior . . . 35
7.5 Image-Based Training Process . . . 35
7.5.1 Learning Visual Similarity from Overlap Metrics . . . 36
7.5.2 Integration into the Word List Solver . . . 36
7.5.3 Impact on OCR Pipeline . . . 36
7.6 Conclusion . . . 36

8 Progress Overview and Conclusion 38
8.1 Summary of the First Defense . . . 38
8.2 Conclusion . . . 38
8.3 Project Progress Overview . . . 39
8.4 Final Remarks . . . 39

1. Introduction

The *OCR Word Search Solver* project is a group initiative whose objective is to automatically solve word search puzzles from scanned or photographed images. More precisely, the goal is to transform a raw image of a word search grid into structured textual data that can be analyzed and solved algorithmically, in a manner comparable to human reasoning.

To achieve this objective, the project brings together several complementary domains of computer science, including image preprocessing, segmentation, pattern recognition, parallel computing, and neural networks. Each of these components plays a critical role in converting raw pixel data into reliable and interpretable information, ultimately allowing the system to detect, recognize, and locate words within the puzzle grid.

During the first defense, the primary focus was on establishing a **solid functional foundation** for the entire system. At that stage, the objective was not full automation, but rather to demonstrate that each individual module of the pipeline could operate correctly, both independently and in coordination with the others.

The first defense validated the following key elements:

- manual image preprocessing, including grayscale conversion, binarization, and manual rotation,
- manual grid and letter localization,
- a fully functional text-based word search solver,
- and an initial neural network proof of concept.

This phase served as an early technical validation of the project architecture, as well as a test of the team's ability to collaborate effectively under strict technical constraints. All components were implemented in **C**, complied with the `C17` standard without compilation warnings, and were integrated within a clean and well-structured codebase.

Building upon these foundations, the second and final phase of the project focused on **automation, optimization, and full system integration**. Several major improvements were introduced to transform the initial proof-of-concept into a robust and efficient OCR application. Manual preprocessing steps were progressively replaced by automated algorithms, including automatic deskewing of the input images. The segmentation and recognition pipeline was extended with multithreaded execution, hardware-accelerated comparison methods, and improved resource management.

In parallel, a graphical user interface was developed to provide visual feedback, allowing detected words to be highlighted directly on the grid and facilitating both user interaction and debugging.

The neural network component was also fully integrated into the pipeline. Rather than remaining an isolated prototype, it now operates as an alternative recognition mode alongside a highly optimized classic method, illustrating the trade-offs between performance-oriented engineering and neural-based recognition.

Throughout the entire project, particular attention was paid to robustness, performance, and modularity. The final system combines low-level image processing, algorithmic logic, parallel execution, and graphical visualization into a coherent and efficient workflow. Careful design choices were made to ensure scalability, predictable performance, and ease of integration between components.

This final defense therefore represents the culmination of the project. It demonstrates not only the technical maturity of the developed OCR system, but also the team's ability to design, implement, optimize, and integrate a complex software solution under realistic constraints.

In the following chapters, we present the final architecture of the system, the detailed implementation of each module, and a quantitative evaluation of the improvements achieved since the first defense.

2. Objectives of the First Defense

According to the official project specification, the first defense (**D1**) aims to validate the core functionality of the OCR pipeline. At this stage, the goal is not full automation yet, but to prove that each component of the process works correctly and integrates within a consistent workflow.

The following modules and features must therefore be **fully functional and demonstrable:**

1. **Image loading and preprocessing**
   - Load a scanned or photographed image.
   - Convert it from color to grayscale, then to binary (black and white) for analysis.
2. **Manual rotation of the image**
   - Allow the user to manually correct the tilt of the image by entering a rotation angle.
3. **Manual localization of key regions**
   - Identify and isolate:
     - The main **grid** containing letters.
     - The **word list** to be found.
     - The individual **letters** inside the grid.
     - The **words** in the list section.
4. **Splitting and saving letters**
   - Segment the grid into individual cells.
   - Save each letter as a separate image file for later recognition.
5. **Word Search Solver (C language)**
   - Implement a C program that reads a text-based grid.
   - Search for given words and output their coordinates (start and end positions).
6. **Neural Network Proof of Concept**
   - Develop a neural network capable of learning the Boolean function:

$$f(A, B) = A \cdot \overline{B} + A \cdot B$$

- Validate its ability to generalize simple logic-based patterns.

In summary, the first defense ensures that every essential step — from image import to basic logic learning — is implemented, tested, and ready for automation in the next project stages.

3. Team Organization and Planning

The success of the *OCR Word Search Solver* project relies on the collaboration of four team members, each responsible for a distinct and complementary module. This distribution allows the team to work efficiently in parallel while ensuring that every component of the pipeline integrates smoothly into a coherent system.

The following table summarizes the members' main roles and deliverables for the first defense (**D1**).

Member	**Main Role**	**Deliverables (D1)**
Eliott	Image preprocessing & rotation	Image loading, grayscale conversion, binarization, and manual deskew for initial image correction. Automated rotation.
Dawid	Localization & letter splitting	Manual ROI (Region of Interest) selection, extraction of grid and word list, and saving each detected letter as an individual image.
Thomas	Solver	Development of a command-line word-search solver in C and display the graphic solver, capable of scanning a text grid in all eight directions and returning the coordinates of found words circling it.
Tiago	Neural network (Proof of Concept)	Implementation of a simple MLP (Multi-Layer Perceptron) that learns the Boolean function $f(A, B) = A \cdot \overline{B} + A \cdot B$, serving as the foundation for later OCR-based letter recognition.

Each member's work directly contributes to building a complete and functional pipeline for the first defense. Eliott and Dawid's modules handle the image preprocessing and segmentation stages, while Thomas's solver and Tiago's neural network begin forming the logical and intelligent components of the system.

This organization ensures a clear separation of tasks while promoting regular collabo-

ration and code integration. The team's coordinated effort during this phase established a solid foundation for the upcoming stages, where full automation and advanced OCR capabilities will be introduced.

4. Image Preprocessing (Eliott)

4.1 Objective

The goal of the **image preprocessing module** is to transform raw scanned inputs into clean, binary, and properly aligned images ready for segmentation. This module includes both manual and automatic deskewing, allowing the system to correct image orientation either through user input or by automatically estimating the skew angle from image content.

This component plays a critical role in the OCR pipeline, as the accuracy of subsequent grid detection, letter localization, and character recognition depends heavily on the quality and geometric consistency of the input images.

The preprocessing module is implemented entirely in C, following the main project constraints (standard C17, low-level memory management, and fixed image dimensions), and serves as a solid foundation for the later OCR stages.

4.2 Personal Experience and Steps Implemented

From the beginning, my objective was to design a precise and efficient rotation module capable of correcting skewed images while preserving both image size and visual quality.

I first implemented a *centered reverse-mapping rotation algorithm*, using precomputed sine and cosine tables in order to reduce redundant trigonometric calculations. This approach ensures stable pixel mapping while avoiding cumulative rounding errors during rotation.

During the early stages, several issues appeared, particularly related to pixel alignment and border clipping, especially for rotations of 45° or 90°. After refining the coordinate transformations and adding strict bounds checking, the output became stable and visually accurate. This resulted in a fast and reliable manual rotation system, well suited for interactive image preprocessing.

In a second step, this work was extended with the implementation of an **automatic deskewing algorithm**. The algorithm analyzes the binarized image to detect black pixel structures corresponding to grid lines. By identifying a reference pixel near the top of the grid and following connected black pixels, the system approximates the global orientation

of the grid. A skew angle is then computed using vector normalization and trigonometric analysis.

One of the main challenges was ensuring that the detected line was representative of the overall grid orientation, despite noise, small artifacts, or partially broken lines. To avoid over-correction, a minimal angle threshold was introduced so that very small detected skews are ignored.

Once the angle is computed, it is passed to the same centered reverse-mapping rotation function used for manual rotation. This design choice avoids code duplication and guarantees consistent behavior between manual and automatic rotation modes.

The final version therefore supports both manual and automatic deskewing, performs efficiently, and integrates seamlessly into the image preprocessing pipeline, providing well-aligned images for the subsequent OCR stages.

4.3 Complexity Analysis

For the image rotation process, the overall computational complexity is dominated by the pixel mapping step, which operates in:

$$O(w \times h)$$

where w and h denote the image width and height, respectively.

In the case of **manual rotation**, trigonometric values (sine and cosine) are precomputed once per row and per column. This results in an additional preprocessing cost of:

$$O(w + h)$$

which avoids redundant trigonometric computations during the pixel iteration phase and significantly improves performance.

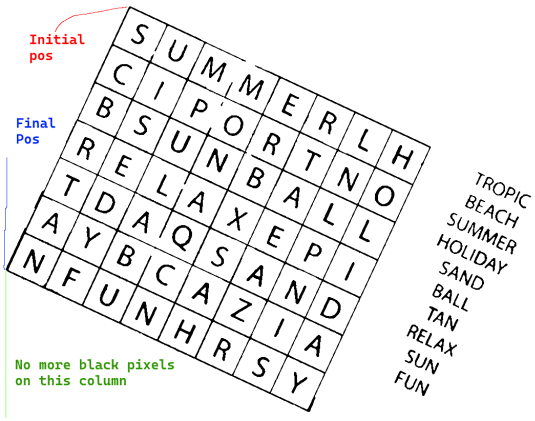
For **automatic deskewing**, the skew detection stage introduces an additional scan of the binarized image in order to estimate the dominant orientation of the grid lines. In the worst case, this detection step also runs in:

$$O(w \times h)$$

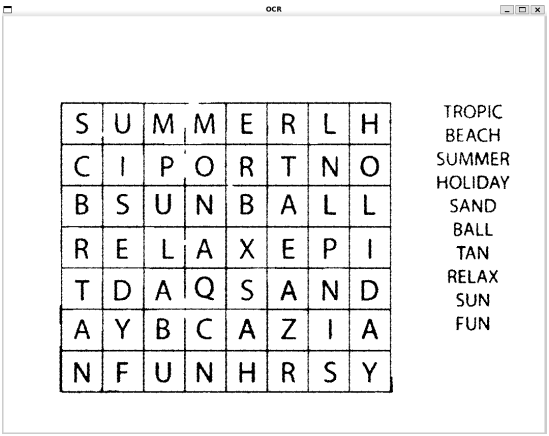
However, its computational cost remains lightweight compared to the rotation itself and does not affect the overall efficiency of the preprocessing pipeline.

Overall, both manual and automatic rotation modes maintain a practical and efficient complexity, making the approach suitable for real-time preprocessing while preserving numerical accuracy and visual stability.

4.4 Example



Before rotation



After rotation

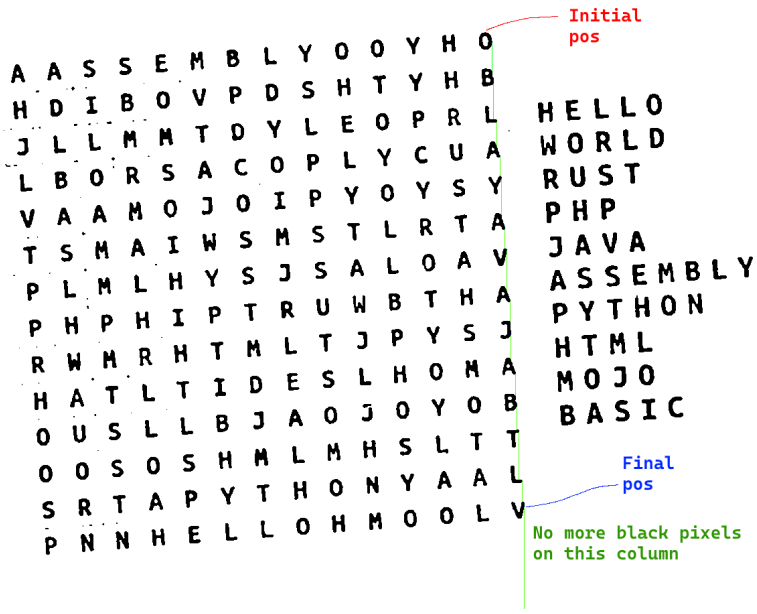


Figure 4.1: Manual/Automated image rotation process implemented by Eliott.

4.5 Integration with the OCR Pipeline

In the final version of the project, the preprocessing module plays a central role in the OCR pipeline by ensuring that all subsequent stages operate on geometrically consistent and well-aligned data.

This module is used to:

- Automatically or manually correct skewed grids before the letter localization stage, depending on image quality and user requirements.

- Guarantee geometric consistency for Dawid's grid localization and letter splitting algorithms, improving segmentation reliability.
- Provide binarized and orientation-stable input data for Tiago's neural network, contributing to more accurate and robust character recognition.

By unifying manual and automatic deskewing within the same preprocessing framework, this module ensures a smooth and reliable transition from raw image input to structured OCR data.

4.6 Personal Reflection

After this preprocessing step, the image is binarized and correctly aligned, making it ready for letter localization and further OCR processing.

Working on both manual and automatic rotation significantly improved my understanding of geometric transformations and optimization techniques applied to low-level image processing. While image rotation may initially appear straightforward, implementing it in a precise and efficient manner revealed several non-trivial challenges.

Ensuring pixel-level accuracy, handling border conditions correctly, and avoiding unnecessary visual distortions required careful mathematical reasoning and iterative testing. The automatic deskewing component, in particular, highlighted the difficulty of translating visual intuition into reliable pixel-based detection and accurate angle estimation.

This work also emphasized the importance of robustness in image preprocessing, as even small alignment errors can propagate through the pipeline and negatively impact later stages such as segmentation and character recognition.

Overall, this module now provides a solid and reliable foundation for the OCR pipeline, ensuring that subsequent components operate on clean, stable, and well-aligned images.

5. Manual Localization & Letter Splitting (Dawid)

5.1 Purpose of the Module

The **letter localization and segmentation module** is responsible for transforming a preprocessed image of the word search puzzle into a structured grid of letters. It serves as the bridge between the visual preprocessing stage and the textual solver, making it the first step toward true OCR functionality.

This component detects the position of the puzzle grid and the associated word list directly from the image. It then removes grid lines, isolates individual characters, and reconstructs a textual representation of the entire puzzle. The process is implemented entirely in **C with SDL2**, providing low-level access to pixel data and full control over the segmentation logic.

Core functions include:

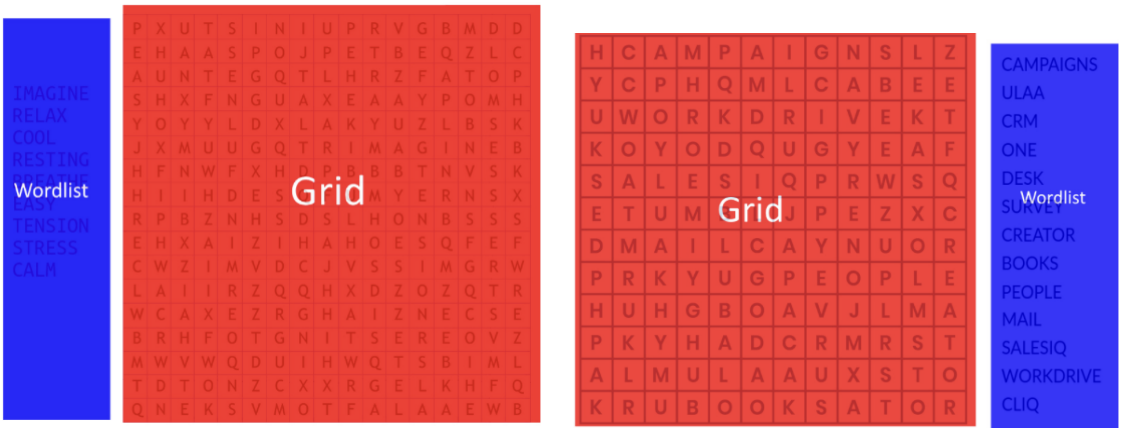
- Automatic localization of the grid and the word list zones
- Removal of grid lines to isolate characters
- Connected-component analysis to extract letters
- Comparison with a reference dataset of letter images (BMP format)
- Generation of a text-based grid for the solver

5.2 Implementation Process

At the start, we focused on designing a stable segmentation pipeline capable of handling imperfect images — including skewed lines, uneven lighting, and small artifacts. Each function was implemented and tested in a modular way, ensuring clarity and ease of debugging.

5.2.1 Grid Detection with `find_grid()`

The function `find_grid()` detects the rectangular area of the puzzle grid. We assume that all images can be divided into two distinct parts — the **grid section** and the **word list section**.

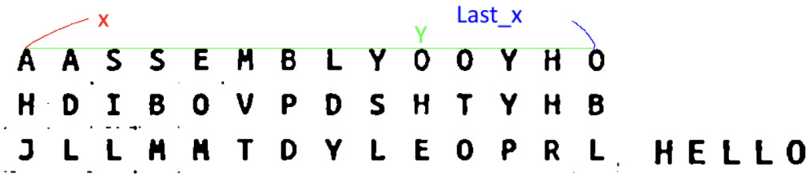


Original tilted grid image *Aligned grid after manual rotation*

Figure 5.1: Comparison between the raw puzzle image (left) and the manually aligned version (right) used for grid localization.

The main assumption is that the grid always starts at a higher vertical position than the word list. The algorithm iterates from top to bottom and left to right, searching for the uppermost black pixel that defines the left and right bounds of the grid.

When searching for `last_x`, the function looks two pixels south of the current position — which means it works better if the grid is rotated downward rather than upward.



Next, the algorithm iterates vertically from y to the bottom of the image and horizontally from x to `last_x`. If at least one black pixel is found at the current y level, it sets `last_y` to that value. It also checks for neighboring black pixels on the left or right boundaries to refine x and `last_x` accordingly.

Once completed, the rectangle dimensions are computed as:

$$\text{width} = \texttt{last_x} - x + 1 \quad \text{and} \quad \text{height} = \texttt{last_y} - y + 1$$

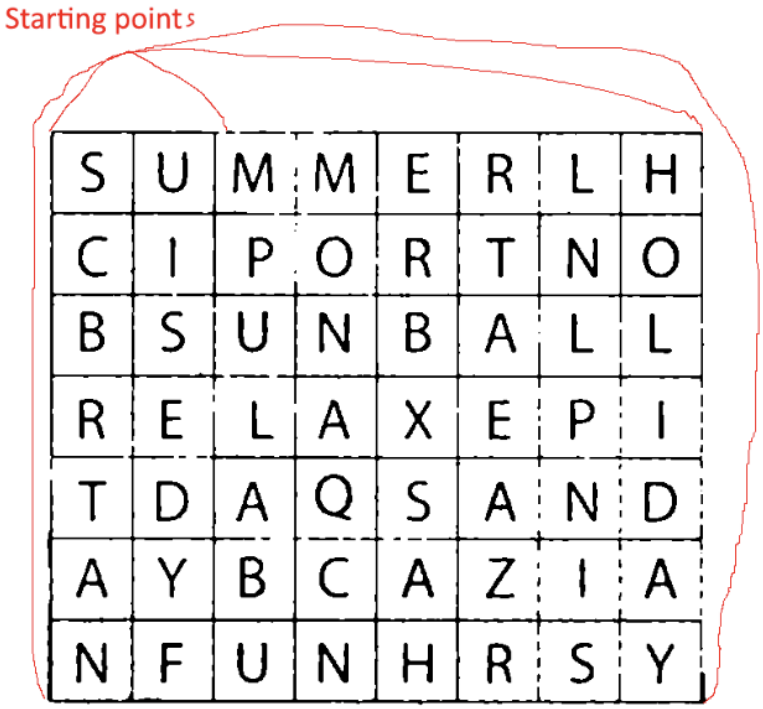
Algorithm conditions for success:

- Solid grid lines are present
- Two "trash" pixels encompass the grid without touching the word list
- Letters start at the same y level with no extra pixels above them

5.2.2 Grid Line Removal with `remove_grid_lines()`

After detecting the grid, the function `remove_grid_lines()` erases both horizontal and vertical lines, leaving only the letters. It uses a **Depth-First Search (DFS)** algorithm with multiple starting points:

- The upper-left black pixel found by `find_grid()`
- The upper-right, lower-left, and lower-right grid corners



After computing the rectangle from the DFS, it is compared with the rectangle found by `find_grid()`. If identical, another DFS is performed to repaint all encountered pixels white — effectively removing the grid lines.

5.2.3 Letter Extraction with `find_all_letters()`

The main segmentation mechanism is implemented in `find_all_letters()`. This function scans the entire grid region from top to bottom, left to right, looking for unvisited black pixels.

When one is found:

1. `get_rect()` isolates the connected black pixel region.
2. The average y position is computed for this region.
3. The region is reset to "unvisited" (using `fill_white()`).
4. The maximum average y is determined to define the next scan line.

If at least one unvisited black pixel remains, a second horizontal loop begins:

- Each pixel region is extracted again using `get_rect()`.
- The rectangle is sent to `find_letter()`, which compares it to 83 reference letters stored in the `res` folder.

`find_letter()` compares 1-bit images using the XOR algorithm. For every pixel difference, a variable `xor_sum` is incremented. The ratio `xor_sum/letter_size` is computed for each reference letter, and the best match (difference $< 25\%$) is chosen.

Some rectangles may be ignored due to unusual dimensions or poor segmentation quality. When a letter is successfully identified, it is appended to an array. After each row scan, the number of detected letters per line is updated.

Each pixel island is then filled with white using DFS (flood fill). For debugging, all detected letters are logged with coordinates and sizes, and the final result is saved as `grid.txt`.

5.2.4 Word List Extraction

The word list extraction follows a similar process. The remaining space in the image (below the grid) is analyzed using `find_all_letters_word_list()`, a modified version of the main extraction function.

The resulting array of `struct str` (null-terminated strings) contains all recognized words from the list, and this data can be directly used by the solver program.

5.3 Multithreaded Architecture Overview

The software is based on a **master–worker multithreading model**. A single main thread is responsible for the global analysis of the image and task coordination, while five worker threads are dedicated to computationally expensive letter recognition tasks.

This architecture allows the program to clearly separate image traversal and letter detection from character recognition, which significantly improves the overall performance of the system.

5.3.1 Responsibilities of the Main Thread

The main thread performs a fast scan of the input image in order to identify potential letters. A letter is defined as an *island of black pixels*, that is, a connected component of black pixels surrounded by white pixels or grid boundaries.

For each detected island, the main thread:

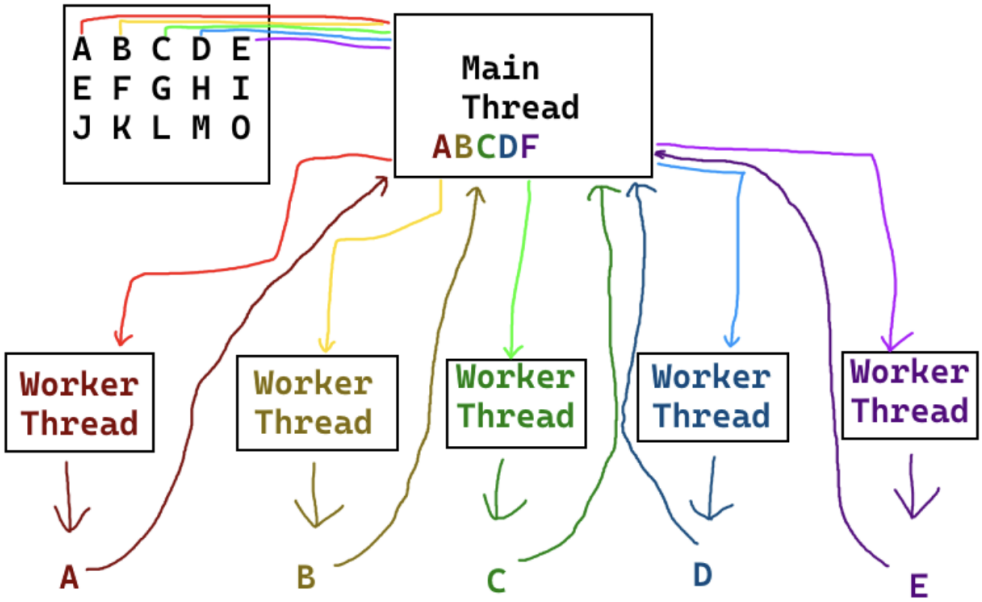
- computes the bounding box of the connected component,
- extracts the corresponding sub-image,
- packages the data required for recognition (bitmap, size, and position).

This step is intentionally lightweight. The main thread does not attempt to classify the letter; its sole responsibility is to determine *what* needs to be recognized and *where* it is located.

5.3.2 Task Delegation to Worker Threads

Once a letter candidate has been extracted, the main thread assigns the recognition task to an available worker thread. If all workers are busy, the task is placed in a queue until a worker becomes free.

Each recognition task is independent, making letter recognition highly parallelizable. This design avoids unnecessary synchronization between worker threads and minimizes contention, resulting in efficient task scheduling.



5.3.3 Worker Thread Processing

Each worker thread operates in a continuous loop:

- waiting for a recognition task,
- processing the extracted letter independently,
- applying the recognition method selected by the user.

Two recognition modes are supported:

- a **classical comparison mode**, based on XOR operations accelerated with AVX instructions,
- a **neural mode**, which relies on a neural network and is significantly more computationally expensive.

After recognition, the worker thread returns the identified character (and optionally a confidence score) to the main thread.

5.3.4 Synchronization and Thread Safety

Synchronization between threads is deliberately kept minimal and limited to:

- task dispatching,
- result collection.

This synchronization is handled using standard `pthreads` primitives, such as mutexes and condition variables. Since worker threads never modify the global image data, the risk of data races is minimal, which simplifies correctness and improves stability.

5.3.5 Performance Impact and Hardware Considerations

In a single-threaded implementation, letter recognition quickly becomes the main performance bottleneck. By offloading recognition to multiple worker threads:

- the main thread continues detecting new letters without blocking,
- multiple letters are recognized simultaneously.

This parallel pipeline results in a significant increase in processing speed, especially for large grids and when using the neural recognition mode. The performance gain scales well with the number of available CPU cores.

The choice of using one main thread and five worker threads is directly motivated by the hardware configuration of the test environment. The school computers are equipped with an **Intel Core i5-14500T**, which provides enough physical cores to efficiently run several CPU-bound tasks in parallel.

The main thread is dedicated to image scanning and task coordination, while the worker threads handle the computationally intensive letter recognition. Limiting the number of worker threads to five avoids CPU oversubscription and excessive context switching, ensuring stable and predictable performance. This configuration allows the program to fully exploit available parallelism while remaining well adapted to the target execution environment.

5.4 Comparison Method

5.4.1 Letter Detection and Recognition Process

Letter recognition is performed by the worker threads based on the islands of black pixels detected and extracted by the main thread. Each island represents a potential letter and is processed independently.

For each received island, a worker thread compares the extracted bitmap against a dataset of **85 reference letter bitmaps**. The objective is to determine which reference bitmap best matches the input island. This exhaustive comparison ensures that all supported characters are considered during the recognition process.

5.4.2 Choice of Comparison Method

The system supports two distinct letter comparison methods, which can be selected by the user:

- **Classic mode**,
- **Neural mode**.

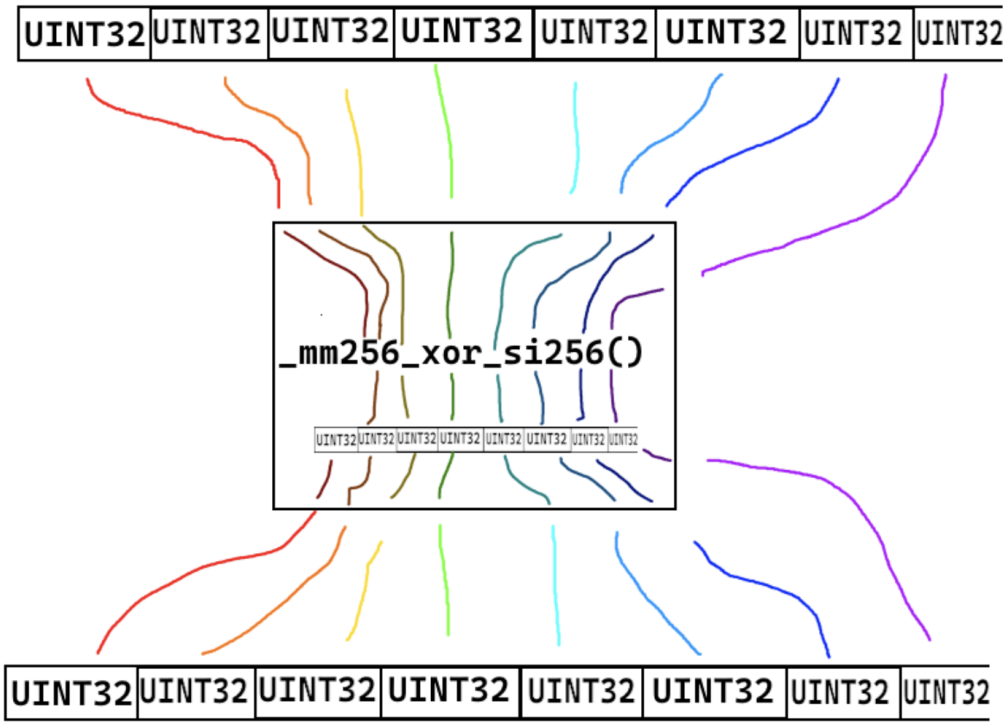
The classic mode is the default and fastest recognition method. The neural mode can be enabled by launching the program with the `-neural` flag. This design choice allows the user to explicitly trade performance for neural-network-based recognition when required.

5.4.3 Classic Comparison Mode

In classic mode, letter recognition relies on a direct pixel-wise comparison between the detected letter island and each bitmap in the reference dataset. The comparison is implemented using a simple bitwise XOR operation between corresponding pixels.

To accelerate this process, the implementation leverages **SSE / AVX2 vector instructions**. Thanks to AVX2, multiple pixels can be compared simultaneously, allowing up to eight pixels to be processed within a single CPU cycle. This results in extremely fast execution and makes the classic mode well suited for large grids and real-time usage.

A limitation of this approach is its hardware dependency. Systems that do not support AVX2 instructions are unable to run the program. For this reason, the target machines used as reference for development and testing were carefully selected, as they fully support AVX2 instructions. This hardware configuration justified the use of such low-level optimizations.



5.4.4 Neural Comparison Mode

The neural mode was introduced primarily to demonstrate that the neural network component is functional and properly trained. It relies on a neural network developed by Tiago and provides an alternative letter recognition strategy.

This method is significantly more computationally expensive and is at least **40 times slower** than the classic XOR-based approach. As a result, it is not intended for performance-critical scenarios, but rather for validation, experimentation, and demonstration purposes.

The neural network is stored in a file named `xor.nw`, located in the `res` directory. Before the letter detection phase begins, the main thread loads and initializes the neural network from this file.

To avoid synchronization issues and potential deadlocks, the initialized neural network is then **cloned for each worker thread**. This ensures that each worker operates on its own private instance of the neural network, eliminating shared-state contention during recognition.

5.4.5 Design Trade-offs

Overall, the two comparison modes reflect a clear design trade-off:

- the **classic mode** prioritizes speed and hardware-specific optimizations,
- the **neural mode** prioritizes flexibility and demonstrates the correctness of the neural-based approach.

This dual-method design highlights both performance-oriented engineering decisions and the successful integration of a neural-based recognition solution within the OCR pipeline.

5.5 Challenges and Solutions

During development, several key challenges emerged:

- **Noise and irregular contrast:** Initial images contained small artifacts mistaken for letters. This was resolved using a median filter and contrast enhancement before segmentation.
- **Memory management:** The recursive nature of flood-fill algorithms required careful memory control. A linked-list-based stack replaced deep recursion to prevent stack overflow.
- **False matches during comparison:** Early tests confused similar characters (e.g., `I` and `T`). This was mitigated by normalizing letter sizes before XOR comparison and setting stricter thresholds.

5.6 Various Optimizations

5.6.1 Resource Loading and Initialization Strategy

In order to perform letter recognition, the program must first load all required resources. These include the dataset of reference letter bitmaps used in the classic comparison mode, as well as the neural network used in the neural mode. Both operations involve disk access and memory allocation, which can become costly if not managed carefully.

To minimize their impact on performance, a dedicated resource loading strategy was implemented, relying on parallelization, caching, and controlled resource lifetime management.

5.6.2 Bitmap Dataset Loading

To retrieve the reference letter dataset, the program reads the bitmap files from storage and converts them into an internal representation suitable for efficient comparison. This loading process is delegated to a dedicated thread, separate from the main execution thread.

By offloading bitmap loading to a separate thread, the program avoids blocking the main execution flow during initialization. Once loaded, the dataset is stored in memory as an array of precomputed bitmaps. This dataset is initialized only once and reused throughout the program's execution.

Keeping the bitmap dataset in memory eliminates repeated loading, deallocation, and reinitialization costs. As a result, subsequent letter detection runs can immediately access the reference data without any additional I/O or memory overhead.

5.6.3 Neural Network Loading and Cloning

A similar approach is applied to the neural recognition mode. The neural network is loaded from disk (from the `xor.nw` file) and initialized by a separate thread, ensuring that this potentially expensive operation does not affect application responsiveness.

Once initialized, the neural network is cloned for each worker thread, so that each thread operates on its own private instance. This design choice prevents shared-state conflicts and eliminates the risk of deadlocks or synchronization issues during neural inference.

5.6.4 Resource Lifetime Management

Unlike the bitmap dataset, neural network instances are not kept permanently in memory. After the recognition phase is completed, neural resources are explicitly released.

This design allows the application to remain flexible. When the user restarts the processing flow using the graphical *back* button, a different neural network file can be loaded during the next execution. By freeing neural resources instead of caching them, the system supports dynamic replacement of neural models without requiring a full application restart.

5.6.5 Design Benefits

This resource management strategy provides several advantages:

- reduced initialization overhead during repeated detections,
- improved responsiveness by offloading I/O operations to separate threads,
- efficient reuse of immutable resources such as the bitmap dataset,
- increased flexibility for neural network experimentation.

Overall, this approach balances performance optimization with modularity and extensibility.

5.6.6 Impact of Optimizations on Performance

The combined use of resource reuse, multithreading, and hardware-specific acceleration significantly improves overall performance. In particular, avoiding repeated resource loading and initialization eliminates unnecessary execution overhead.

These optimizations are especially noticeable after the first execution. While the initial run includes all setup costs and takes approximately **29 ms**, subsequent runs benefit from cached resources and pre-initialized data structures. As a result, the processing time for the first image of the second level is reduced to approximately **19 ms**.

This performance improvement demonstrates the tangible impact of careful optimization and efficient resource management, even when processing identical input data.

5.7 Comparison with the Program Presented in the First Defense

5.7.1 Performance Evaluation and Comparison

With all optimizations in place, the final program achieves a significantly higher execution speed compared to the version presented during the first practical session. To evaluate performance objectively, the optimized version was compared with the original implementation.

All tests were conducted on a laptop equipped with an **Ryzen 7 7435HS** processor and **16 GB of DDR5 RAM**.

Level	Image	Old Version (ms)	New Classic (ms)	New Neural (ms)
1	1	2109	85 → 72	4268 → 4023
1	2	1386	45 → 36	2237 → 2148
2	1	692	29 → 19	1001 → 836
2	2	1620	49 → 42	3366 → 2258

Table 5.1: Execution time comparison between the old program and the optimized versions

5.7.2 Notes on the Results

The original program relies on a simple XOR-based comparison without advanced optimizations such as multithreading, AVX2 acceleration, or resource caching.

In the optimized implementation, the values shown after the arrow (→) represent execution times measured after resource initialization, that is, once the bitmap dataset or neural network has already been loaded into memory. This explains the noticeable performance difference between the first execution and subsequent runs.

The classic mode benefits from hardware-accelerated XOR comparisons and resource caching, resulting in a drastic reduction in execution time compared to the old version.

The neural mode, although inherently slower due to the computational cost of neural inference, also shows substantial performance improvements thanks to multithreading and pre-loading and cloning of the neural network for each worker thread.

5.7.3 Observations

Several key observations can be drawn from these results:

- The classic mode achieves execution time reductions exceeding **95%** in some cases (e.g., Level 1 Image 1: 2109 ms → 72 ms).

- The neural mode, while slower than the classic mode, still benefits significantly from the applied optimizations (e.g., Level 2 Image 2: 3366 ms → 2258 ms).
- Resource initialization and caching play a critical role in performance improvement, as repeated tasks become substantially faster after the first execution.

Overall, these results clearly demonstrate that the optimized program is **orders of magnitude faster** than the original implementation, while still supporting both classic and neural letter recognition methods.

5.8 Demonstration Example

When executed on a sample puzzle image, the module produces the following output:

```
Found grid: x=132 y=184 w=598 h=601
Removed grid lines
letter_position_rect x:150 y:195 w:20 h:22 → A
letter_position_rect x:175 y:195 w:18 h:22 → B
letter_position_rect x:198 y:195 w:19 h:21 → C
...
Letter extraction complete: 25 letters detected
Grid saved as grid.txt
```

The output file `grid.txt` reconstructs the grid in a format directly usable by the solver:

```
A B C D E
F G H I J
K L M N O
P Q R S T
U V W X Y
```

This file can then be used as input for the solver module (Thomas).

5.9 Integration Within the OCR Pipeline

This module forms a crucial intermediary between image analysis and textual interpretation. It takes as input the binarized image generated by Eliott's preprocessing functions and produces:

1. The textual grid (`grid.txt`) to be solved by Thomas's algorithm.
2. Individual letter images for Tiago's neural network training.

It therefore completes the first end-to-end OCR loop:

$$\text{Image} \rightarrow \text{Segmentation} \rightarrow \text{Text Grid} \rightarrow \text{Solver Result}$$

This proves that the pipeline can successfully translate visual data into structured text.

5.10 Next Development Steps

For the second defense, the focus will shift toward automation and robustness:

- Automatic grid and word list detection using the Hough transform
- Morphological filters for better character separation
- Integration of a lightweight neural recognizer
- Full puzzle reconstruction with visualization

5.11 Reflection

This stage was essential in establishing the technical foundations of the entire OCR project. Through direct pixel manipulation and algorithmic reasoning, we built a fully functional letter extraction system capable of feeding both the solver and the neural network.

Beyond immediate functionality, this work also laid the structure for future automation and recognition improvements. It demonstrates how manual control combined with algorithmic logic can yield reliable OCR preparation — a necessary milestone before reaching full automation.

6. Word Search Solver (Thomas)

6.1 Objective

During the first defense (D1), the solver produced a purely textual output: it returned the coordinates of the first occurrence of a word inside a grid, in any of the eight directions.

For Defense 2 (D2), the goal was to go one step further by adding a **graphical interface** using **SDL2**. The new module makes the solver results immediately understandable by the user by:

- displaying the **letter grid** visually,
- rendering the **word list** on the side,
- and especially **highlighting the found words directly on the grid**.

This stage transforms the solver from a command-line tool into an actual *visual validation component* of the OCR pipeline.

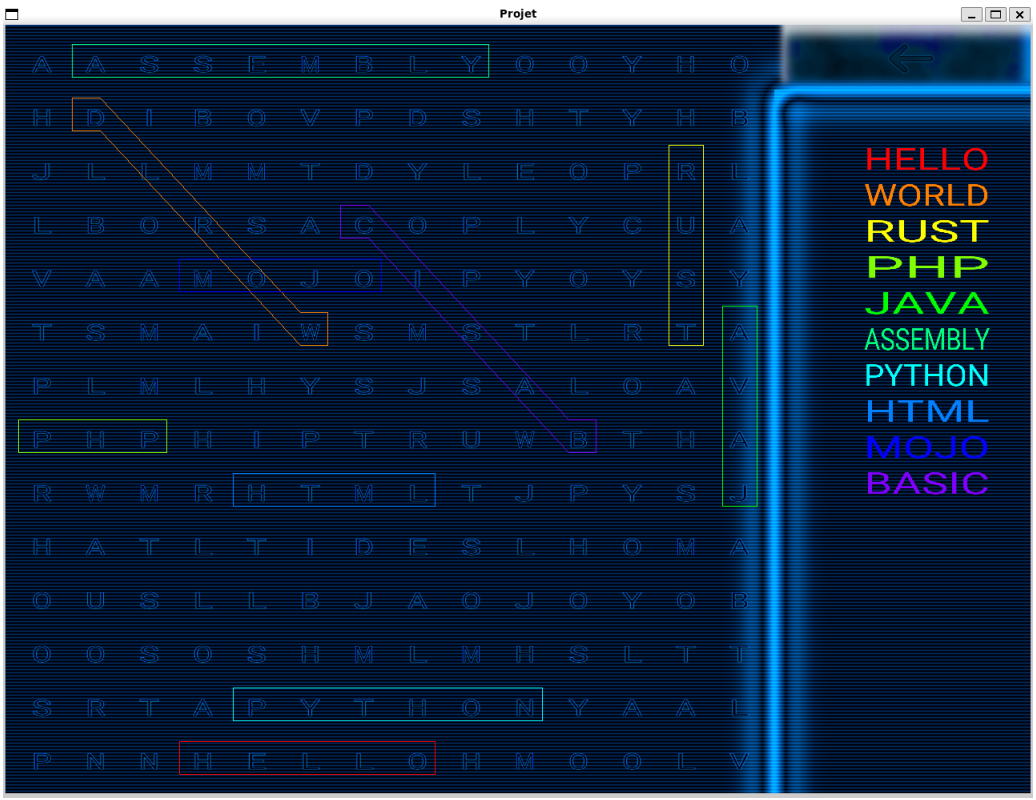


Figure 6.1: D2 graphical solver: grid rendering + word list + highlighted solutions.

6.2 Personal Work and Experience

From the beginning, my objective was to create a robust, efficient, and compliant word solver that could easily integrate into the larger OCR pipeline. As we saw on the first defense the result was a success.

To do a recapitulation :

I started by analyzing the problem algorithmically:

> Given a grid of letters, find a word by exploring all 8 directions from each cell that matches the first letter.

This approach is simple in concept but required careful implementation to handle edge cases such as:

- Grid boundaries (avoiding out-of-range access)
- Letter case normalization
- Irregular grid sizes or missing columns

During the first iterations, I faced several issues:

- The parser was not handling newline characters properly when reading the grid.
- Some words were being found twice due to incorrect boundary conditions.
- I also had to standardize letter casing using `toupper()` to avoid mismatches from the OCR outputs.

Once these bugs were resolved, the solver became stable and produced correct results across all test grids.

Now the next step was to code the graphic interface. My main objective in D2 was to create a clean visualization layer on top of the solver, while keeping the constraints of the project: **C17**, no warnings, stable memory management, and portability.

At first, the challenge looked simple: "draw the grid and draw a line on the result". In practice, building a correct SDL renderer required solving several non-trivial problems: coordinate conversions, scaling, texture handling, font rendering, and avoiding leaks.

The work done in this phase allowed me to move from **algorithmic thinking** to **system + interface thinking**, which is essential for the final integrated OCR.

6.3 Implementation and Technical Additions

The D2 implementation is mainly contained in `graphical_solver.c/.h`, which introduces a complete SDL pipeline: resource loading, rendering loop, text display with `SDL_ttf`, and visual marking of solver results.

6.3.1 Resource Loading: Textures for Letters and UI Elements

A first step was to build a reusable texture system.

- **Letters as textures:** the 26 grid letters are loaded into an array of textures (e.g., `g_letters[26]`) from `res/grid_letter/a.png` to `z.png`. This ensures consistent rendering and avoids reloading during execution.
- **UI assets:** a background texture is rendered first, then optional UI textures (buttons, icons) are prepared with alpha blending (`SDL_SetTextureBlendMode`) to allow transparency.

This structure makes the interface scalable: future defenses can easily add more GUI elements without rewriting the core renderer.

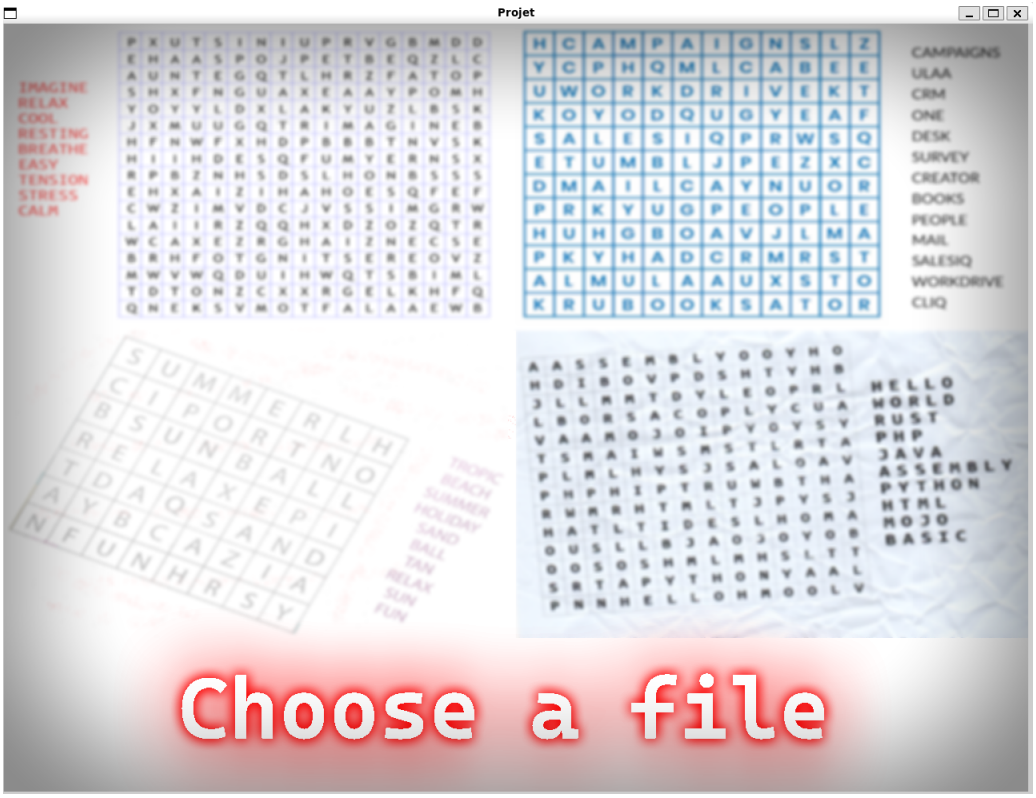


Figure 6.2: Example of UI assets used in SDL2 (background + buttons + letter textures).

6.3.2 Grid Rendering: Scaling from Grid Coordinates to Pixel Coordinates

A key technical point was mapping a logical grid (rows/columns) to screen pixels.

The renderer computes the size of a cell depending on the grid dimensions:

$$\texttt{letter_w} = \frac{940}{\texttt{grid_w}} \qquad \texttt{letter_h} = \frac{940}{\texttt{grid_h}}$$

Each letter is then drawn in its cell using a computed `SDL_Rect`, with small adjustments to keep the glyph centered and readable.

This is important because a simple "raw blit" results in misaligned letters or inconsistent spacing, especially when grid sizes change.

6.3.3 Word List Rendering using `SDL_ttf`

To display the word list clearly, I used **SDL_ttf**:

- load a font (`TTF_OpenFont`),
- render each word into a surface (`TTF_RenderText_Blended`),
- convert it into a texture and draw it in the side panel.

Each word is displayed with a color selected from a palette (modulo indexing), so that the word list visually matches the highlight color on the grid. This improves readability when multiple words are found.

6.3.4 Highlighting Found Words on the Grid

The central addition in D2 is the ability to **draw a visible mark** around the solver result.

The solver outputs the start and end positions of each found word. In the graphical module, these coordinates are converted into pixel space by multiplying them by `letter_w` and `letter_h`.

Then, using `SDL_RenderDrawLine`, the program draws a set of line segments that form a **frame-like highlight** around the word direction. The code handles multiple directional cases:

- left-to-right words (`x < tox`),
- right-to-left words (`x > tox`),
- vertical words (same `x`, with `toy > y` or the opposite),
- and diagonal directions through the same start/end pixel mapping logic.

The goal is not only to "show a result", but to provide an immediate visual confirmation: the user can verify that OCR extraction + solver logic matches the actual puzzle layout.

6.3.5 Graphical Style and Visualization Choices

Beyond the functional aspects of the graphical solver, particular attention was given to the choice of the visual style used to display the grid and highlight the solutions.

The main objective was to design a visualization that remains clear, readable, and informative, without introducing unnecessary visual complexity. For this reason, a minimalistic and functional graphical style was adopted.

Letters are rendered using simple, high-contrast textures to ensure good readability regardless of grid size. The background remains neutral, allowing the grid and highlighted words to stand out clearly. Rather than using filled shapes or opaque overlays, solutions are highlighted using thin line-based frames drawn directly around the detected words.

This design choice provides several advantages:

- it preserves the visibility of the underlying letters,
- it avoids hiding potential OCR or segmentation errors,
- and it allows multiple words to be highlighted simultaneously without cluttering the interface.

Colors are used sparingly and consistently. Each word in the word list is assigned a distinct color from a predefined palette, and the same color is reused to highlight the corresponding word on the grid. This visual correspondence makes it easy to associate each solution with its textual representation.

Overall, the chosen graphical style prioritizes clarity, robustness, and debugging efficiency. It supports both demonstration purposes and technical validation, making the graphical solver a practical tool for analyzing and verifying OCR results.

6.4 Challenges and Solutions

1) Coordinate Conversion (Grid vs Pixels)

The most frequent source of errors was coordinate mismatch. A small offset mistake (for example using `letter_h » 2` incorrectly) immediately shifts the highlight and makes it miss the letters.

Solution: I standardized the coordinate pipeline: grid index → pixel coordinate → drawing offsets, and tested multiple grid sizes to validate stability.

2) Readability Under Scaling

Different grid dimensions mean different cell sizes. Some letters were not centered correctly, and some glyphs looked too thin or too large.

Solution: I introduced slight rectangle adjustments (custom `temp_rect`) and special handling for narrow letters when needed (e.g. `I`), ensuring consistent visual alignment.

3) SDL Resource Management

SDL introduces explicit memory responsibilities: `SDL_Surface`, `SDL_Texture`, and `TTF_Font` must all be released properly to avoid memory leaks.

Solution: I enforced a strict lifecycle: `Create` → `Use` → `Destroy`, and verified cleanup paths even on error exits.

4) UI/UX Consistency

A correct highlight is not enough if the interface is confusing. With many words, the user needs clear visual pairing between the list and the grid.

Solution: I implemented a color palette and applied the same color to both the text word and its grid highlight.

6.5 Integration with the OCR Pipeline

This graphical module plays a direct role in the global system:

- it receives the grid generated by the OCR/segmentation stage,
- it displays the recognized word list,
- it renders the solver results visually, enabling fast validation and debugging.

As a result, D2 adds a missing feedback loop:

OCR Output → Solver → **Graphical Validation (SDL)**

This helps detect recognition errors early (misread letters, incorrect grid extraction), because the user can see mismatches instantly.

7. Neural Network Proof of Concept (Tiago)

7.1 Introduction

The purpose of this component was to design a simple yet functional neural network system capable of performing logical operations (**XOR** and **AND**) directly on image data, and then extend it to evaluate similarity between two images — a foundational concept in optical character recognition (OCR).

This part of the project demonstrates the use of artificial intelligence to compare and interpret visual data, laying the groundwork for the future OCR automation stages. The model was coded entirely in **C**, without external libraries, using only mathematical primitives and custom data structures.

7.2 System Architecture

The neural system is composed of three main parts:

1. **Neural Network Core:**
   Implements a feedforward, fully connected neural network with:
   - Sigmoid activation function
   - Backpropagation with learning rate control
   - Modular, layered neuron structure
2. **Training Process:**
   Handles:
   - Logical operation learning (XOR, AND) on pixel-based inputs
   - Similarity metric computation (IoU, Dice coefficient)
   - Learned similarity mapping via a dedicated neural model
3. **Main Application:**
   Provides a command-line interface allowing:
   - Dynamic image loading (`./xor_nn <imageA.bmp> <imageB.bmp>`)
   - Real-time network training and inference
   - Output of XOR and AND images
   - Computation of similarity metrics and learned similarity score

7.3 Algorithms Used

7.3.1 Neural Network Forward and Backpropagation

The model uses the sigmoid activation function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

and its derivative:

$$\sigma'(y) = y(1 - y)$$

During forward propagation, each neuron's output is computed as:

$$\text{output}_i = \sigma\left(\sum_j w_{ij} x_j + b_i\right)$$

Weights are updated through gradient descent using:

$$\Delta w_{ij} = \eta \cdot \delta_i \cdot x_j$$

where η is the learning rate and δ_i is the error propagated backward through the layers.

7.3.2 Logical Operation Learning

Two separate neural networks are trained to reproduce the truth tables of XOR and AND:

$$f_{XOR}(x, y) = (x \oplus y) \qquad f_{AND}(x, y) = (x \wedge y)$$

Each pixel pair (A_i, B_i) from the two input images is converted into binary inputs (0 or 1). The target output is defined by the corresponding logical function. Through multiple epochs, the network learns the decision boundary separating the true and false outputs of each operation.

7.3.3 Learned Similarity Network

A third neural network — the **Similarity Net** — maps pairs of overlap metrics ($IoU, Dice$) to a continuous similarity score between 0 and 1. It is trained on a small synthetic dataset:

IoU	Dice	Target Similarity
1.00	1.00	1.00
0.85	0.90	0.90
0.70	0.80	0.70
0.50	0.65	0.50
0.30	0.45	0.25
0.10	0.20	0.05

This model generalizes the relationship between geometric overlap and perceived similarity — a principle that underlies OCR pattern matching.

7.4 Final System Behavior

When executed as:

```
./xor_nn a.bmp b.bmp
```

the system performs:

1. Neural XOR and AND operations.
2. Computation of overlap metrics (IoU and Dice).
3. Learned similarity inference.

The output includes both numerical and visual results:

```
=== RESULTS ===
IoU:  0.6782
Dice: 0.8082
Neural Similarity Score: 0.755
```

and saves the following output images:

- `xor_output.bmp`
- `and_output.bmp`

7.5 Image-Based Training Process

Unlike conventional neural networks trained on abstract numerical datasets, the proposed model is trained directly on image data at the pixel level. Each training sample corresponds to a pair of pixels extracted from two aligned binary images. Prior to training, images are converted to grayscale and binarized using adaptive or fixed thresholds, ensuring that each pixel is represented as a normalized value in the range $[0, 1]$.

For each pixel position (i), the corresponding values from the two images (A_i, B_i) are fed as inputs to the neural network. Depending on the task, the target output is defined according to the desired logical operation. For XOR training, the target is set to 1 if exactly one of the pixels is active, and 0 otherwise. For AND training, the target is 1 only when both pixels are active. This pixel-wise supervision allows the network to learn logical behavior directly from visual data.

Training is performed using standard backpropagation with a sigmoid activation function. Weights and biases are updated after each pixel sample, effectively treating each pixel pair as an independent training instance. This dense supervision enables rapid convergence even with relatively small images and limited training epochs.

To compensate for class imbalance in the AND operation (where positive samples are rare), A weighted error term is applied during training. Positive targets receive a higher update weight, ensuring that overlapping foreground regions are learned accurately.

7.5.1 Learning Visual Similarity from Overlap Metrics

We previously used a similarity network to compute similarity between images. However we have now abandoned it. Opting only for the previous two networks

7.5.2 Integration into the Word List Solver

The neural network module is integrated into the word list solver as a visual comparison engine. During word recognition, segmented character images extracted from the input grid are compared against reference character templates derived from the word list. For

each candidate comparison, the system performs the following steps:

1. Preprocessing and binarization of the segmented character and reference template.
2. Pixel-wise neural XNOR operations to highlight differences and overlaps.
3. Chooses the most similar image.

The resulting similarity score represents how closely the segmented character matches a given template. This score is used by the word list solver to rank candidate letters and prune implausible matches. By aggregating similarity scores across multiple characters, the solver can efficiently evaluate entire word candidates.

This neural integration transforms the word list solver from a purely rule-based system into a hybrid pipeline combining classical image processing with learned visual inference. As a result, the solver becomes more tolerant to distortions, segmentation errors, and handwritten or imperfect input images.

7.5.3 Impact on OCR Pipeline

The integration of neural image comparison constitutes a key step toward end-to-end OCR. By learning logical operations and similarity measures directly from visual data, the system avoids reliance on handcrafted heuristics. This approach improves robustness and lays the foundation for future extensions, such as full character classification and sequence-based word recognition.

Overall, the neural module acts as the decision-making core of the word list solver, bridging low-level pixel data and high-level lexical reasoning.

7.6 Conclusion

This neural network module successfully demonstrates the feasibility of learning logical and similarity operations directly from visual data. It acts as a conceptual bridge between basic mathematical learning and OCR recognition, proving that neural models can interpret and compare patterns in a meaningful way.

Future work will extend this foundation toward:

- Character recognition (A–Z classification)
- Integration with Dawid's segmentation module
- End-to-end OCR inference for full word reconstruction

This marks a major step forward in connecting the project's image-processing components with artificial intelligence capabilities.

8. Progress Overview and Conclusion

8.1 Summary of the First Defense

The first defense validates all mandatory milestones of the project, confirming that each component of the OCR Word Search Solver functions as intended. Throughout this phase, the team successfully implemented and demonstrated the core modules that form the foundation of the complete system:

- **Preprocessing:** manual grayscale conversion, binarization, and rotation.
- **Localization:** manual region of interest (ROI) selection and letter extraction.
- **Solver:** fully functional C program capable of searching words in eight directions.
- **Neural Network:** successful proof of concept learning a Boolean logic function.

At this stage, the pipeline performs a complete manual OCR preparation and text-solving loop, demonstrating the feasibility and robustness of the overall system. The integration of all modules has validated not only the technical objectives but also the team's ability to collaborate effectively under shared constraints.

8.2 Conclusion

The final defense marks the completion of the *OCR Word Search Solver* project and validates the transition from a functional proof-of-concept to a nearly fully automated and integrated system.

During this phase, the project evolved significantly from the first defense: manual steps were progressively replaced by automated processing, the solver was extended with a graphical interface, and the neural network component reached full operational capability for OCR tasks.

The main objectives of this final stage were therefore achieved:

- full integration of the image processing, segmentation, solver, and GUI modules,
- improved robustness and accuracy in letter recognition,

- visual validation of solver results through an SDL-based interface,
- and a coherent end-to-end pipeline, from image input to solved puzzle display.

8.3 Project Progress Overview

The table below summarizes the overall progress of the project between the first defense and the final defense. It highlights the significant advancements made during the second phase, particularly regarding the graphical interface, the solver, and the neural network.

Task	**Phase 1 : First Defense (Nov 5th)**	**Phase 2 : Final Defense (Dec 17th)**
Image treatment	100%	100%
GUI	20%	90%
Neural Network	70%	100%
Solver	40%	100%
Rotation	50%	100%

Table 8.1: Project progress between the first and final defenses

8.4 Final Remarks

This final defense confirms that all core objectives of the project have been met. The system now relies on a solid technical foundation and a well-structured architecture that connects image processing, optical character recognition, algorithmic solving, and graphical visualization.

Each team member contributed a critical and complementary component, resulting in a unified and functional OCR pipeline. Beyond the technical achievements, the project also demonstrated effective collaboration, clear task distribution, and successful integration under strict implementation constraints.

With these results, the project successfully completes its intended scope. It represents a transition from experimental development to a robust and demonstrable OCR application, capable of solving word search puzzles in a reliable and visually verifiable manner.